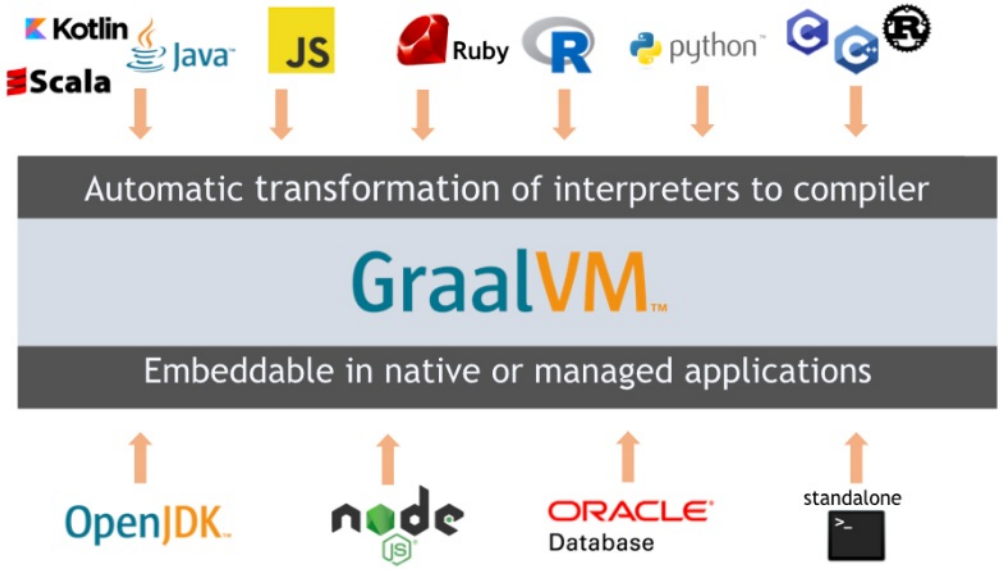
Paradigme de Programare - Laborator 2

Utilizarea limbajului PolyGlot cu GraalVM

GraalVM suportă, după cum am discutat, apeluri de instrucțiuni aparținând mai multor limbaje. În exemplul de mai jos avem o aplicație schelet (template) care, utilizând un tablou de cuvinte, va calcula o sumă de control și apoi afișa cuvintele împreună cu suma de control calculată pentru acestea. Acest lucru se poate face în orice limbaj, însă aici întâlnim utilizarea simultană a patru limbaje diferite.



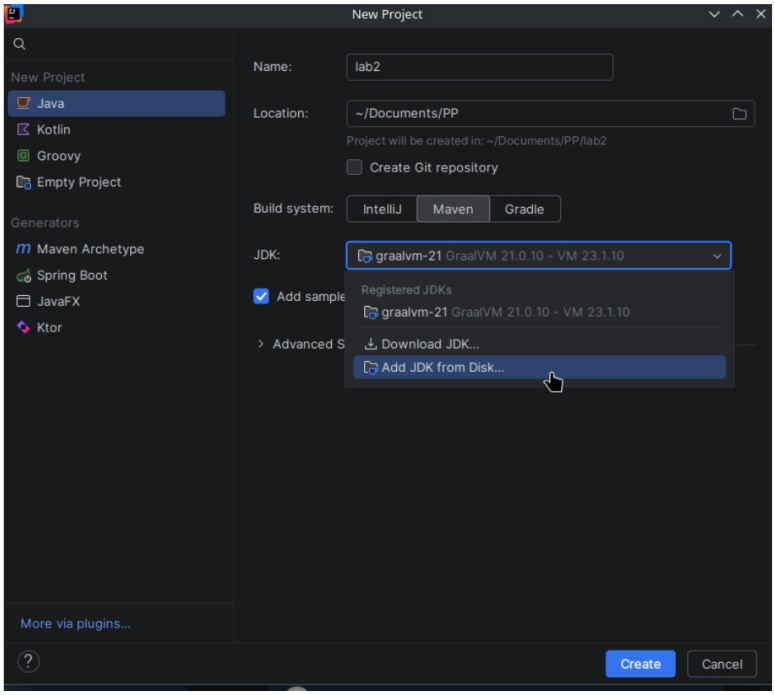
Sursa: https://www.graalvm.org/docs/introduction/

Instalarea și configurarea GraalVM

Descarcati GraalVM 25, Linux x64 de aici:

https://www.graalvm.org/downloads/#

Dezarhivati, apoi in Intellij, in fereastra de New Project, selectati **Maven** pentru Build System si selectati folder-ul cu GraalVM pentru JDK:



Apoi apasati pe **Create.**

In fisierul Main.java puneti codul exemplu.

```
package org.example;

import org.graalvm.polyglot.Context;
import org.graalvm.polyglot.Value;

import java.util.Set;

public class Main {
  public static void main(String[] args) {
    try (Context context = Context.newBuilder().allowAllAccess(true).build()) {
      Set<String> languages = context.getEngine().getLanguages().keySet();
      for (String id : languages) {
        System.out.println("Initializing language " + id);
        context.initialize(id);
        switch (id) {
          case "python":
            context.eval("python", "print('Hello Python!')");
            break;
          case "js":
            context.eval("js", "print('Hello JavaScript!');");
            break;
          case "ruby":
            context.eval("ruby", "puts 'Hello Ruby!'");
            break;
          case "java":
            Value out = context.getBindings("java").getMember("java.lang.System").getMember("out");
            out.invokeMember("println", "Hello Espresso Java!");
            break;
        }
      }
    }
  }
}
```

Codul este menit sa arate cum puteti executa cod din diferite limbaje.

In fisierul pom.xml trebuie sa adaugati versiunea de Graal folosita si dependencies ce instaleaza polyglot-ul si limbajele dorite.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>lab2</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>25</maven.compiler.source>
    <maven.compiler.target>25</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <graalvm.version>25.0.2</graalvm.version> <!-- ADAUGAT -->
  </properties>

  <!-- ##### ADAUGAT! ##### -->
  <dependencies>
    <dependency>
      <groupId>org.graalvm.polyglot</groupId>
      <artifactId>polyglot</artifactId>
      <version>${graalvm.version}</version>
      <type>jar</type>
    </dependency>
    <dependency>
      <groupId>org.graalvm.polyglot</groupId>
      <artifactId>js</artifactId>
      <version>25.0.2</version>
      <type>pom</type>
      <scope>compile</scope>
    </dependency>
    <dependency>
      <groupId>org.graalvm.polyglot</groupId>
      <artifactId>python</artifactId>
      <version>25.0.2</version>
      <type>pom</type>
      <scope>compile</scope>
    </dependency>
    <dependency>
      <groupId>org.graalvm.polyglot</groupId>
      <artifactId>ruby</artifactId>
      <version>25.0.0</version>
```

```
      <type>pom</type>
      <scope>compile</scope>
    </dependency>
    <dependency>
      <groupId>org.graalvm.polyglot</groupId>
      <artifactId>java-community</artifactId>
      <version>25.0.2</version>
      <type>pom</type>
      <scope>compile</scope>
    </dependency>
  </dependencies>

  <!-- ################## -->

</project>
```

Din nefericire limbajul R nu mai are suport in GraalVM Polyglot, asa ca puteti folosi alt limbaj pentru cerintele din laborator.

Instalarea limbajelor suplimentare pentru GraalVM

Dacă încercați să executați aplicația și primiți o eroare de acest tip:

```
Exception in thread "main" java.lang.IllegalArgumentException: A language with id 'R' is not installed. Installed languages are: [js, llvm].
    at com.oracle.truffle.polyglot.PolyglotEngineImpl.requirePublicLanguage(PolyglotEngineImpl.java:853)
    at com.oracle.truffle.polyglot.PolyglotContextImpl.requirePublicLanguage(PolyglotContextImpl.java:842)
    at com.oracle.truffle.polyglot.PolyglotContextImpl.eval(PolyglotContextImpl.java:813)
    at org.graalvm.polyglot.Context.eval(Context.java:344)
    at org.graalvm.polyglot.Context.eval(Context.java:370)
    at Polyglot.RToUpper(Polyglot.java:12)
    at Polyglot.main(Polyglot.java:37)
```

înseamnă că nu aveți limbajul indicat de eroare instalat în SDK-ul GraalVM.

Pentru a instala un limbaj nou, deschideți un terminal în folder-ul GraalVM (în laborator, /home/student/opt/graalvm-ce-java11-22.0.0.2) și executați comanda:

```
./bin/gu install <NumeLimbaj>
```

Pentru acest laborator, veți avea nevoie de R și Python instalate suplimentar:

```
./bin/gu install R
./bin/gu install Python
```

```
Downloading: Release index file from oca.opensource.oracle.com
Downloading: Component catalog for GraalVM Enterprise Edition 22.0.0.1 on jdk11 from oca.opensource.oracle.com
Downloading: Component catalog for GraalVM Enterprise Edition 22.0.0 on jdk11 from oca.opensource.oracle.com
Downloading: Component catalog from www.graalvm.org
Processing Component: FastR
Processing Component: LLVM.org toolchain
```

Aplicație demonstrativă

1. Aplicația principală este scrisă în Java.

```
28      //functia MAIN
29      public static void main(String[] args) {
30          //construim un context pentru evaluare elemente JS
31          Context polyglot = Context.create();
32          //construim un array de string-uri, folosind cuvinte din pagina web:  https://chrisseaton.com/truffleruby/te
33          Value array = polyglot.eval( languageId: "js",  source: "[\"If\",\"we\",\"run\",\"the\",\"java\",\"command\",\"
34          //pentru fiecare cuvant, convertim la upcase folosind R si calculam suma de control folosind PYTHON
35          for (int i = 0; i < array.getArraySize();i++){
36              String element = array.getArrayElement(i).asString();
37              String upper = RToUpper(element);
38              int crc = SumCRC(upper);
39              System.out.println(upper + " -> " + crc);
40          }
41      }
42  }
```

2. Tabloul de cuvinte este definit utilizând JavaScript.

```
32      //construim un array de string-uri, folosind cuvinte din pagina web:  https://chrisseaton.com/truffleruby/tenthing
33      Value array = polyglot.eval( languageId: "js",  source: "[\"If\",\"we\",\"run\",\"the\",\"java\",\"command\",\"incl
```

3. Înainte de calculul sumei de control, cuvintele sunt transformate cu litere mari (upper case) folosind R.

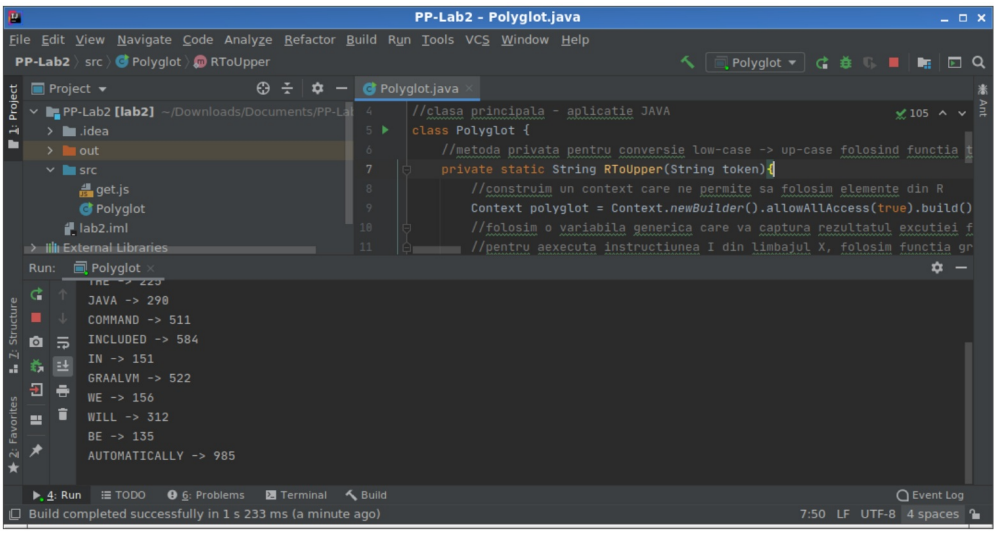
```
7       private static String RToUpper(String token){
8           //construim un context care ne permite sa folosim elemente din R
9           Context polyglot = Context.newBuilder().allowAllAccess(true).build();
10          //folosim o variabila generica care va captura rezultatul excutiei functiei R, toupper(String)
11          //pentru aexecuta instructiunea I din limbajul X, folosim functia graalvm polyglot.eval("X", "I");
12          Value result = polyglot.eval( languageId: "R",  source: "toupper(\"+token+"\");");
13          //utilizam metoda asString() din variabila incarcata cu output-ul executiei pentru a mapa valoarea generica la un String
14          return result.asString();
15      }
```

4. Calculul sumei de control se realizează în Python.

```
17      //metoda privata pentru evaluarea unei sume de control simple a literelor unui text ASCII, folosind PYTHON
18      private static int SumCRC(String token){
19          //construim un context care ne permite sa folosim elemente din PYTHON
20          Context polyglot = Context.newBuilder().allowAllAccess(true).build();
21          //folosim o variabila generica care va captura rezultatul excutiei functiei PYTHON, sum()
22          //avem voie sa inlocuim anumite elemente din scriptul pe care il construim spre evaluare, aici token provine din JAVA, dar va fi
23          Value result = polyglot.eval( languageId: "python",  source: "sum(ord(ch) for ch in '" + token + "')");
24          //utilizam metoda asInt() din variabila incarcata cu output-ul executiei, pentru a mapa valoarea generica la un Int
25          return result.asInt();
26      }
```

Atenție: dacă executați aplicația din IntelliJ în laborator, modificați vectorul array de pe linia 41 pentru a nu conține mai mult de 5 cuvinte. Altfel, stațiile din laborator vor rămâne fără memorie RAM în timpul execuției.

Aplicația în execuție:



Compilarea și execuția aplicației GraalVM din linie de comandă

Dacă executați aplicația din terminal, puteți utiliza mai multe cuvinte în vectorul **array**. Deschideți un terminal în folder-ul care conţine fişierul sursă *Polyglot.java* din proiectul aplicaţiei demonstrative, apoi introduceți următoarea comandă pentru a compila programul folosind GraalVM SDK:

```
/home/student/opt/graalvm-ce-java11-22.0.0.2/bin/javac Polyglot.java
# iar pentru a executa aplicatia:
/home/student/opt/graalvm-ce-java11-22.0.0.2/bin/java Polyglot
```

Instrumente de depanare și monitorizare

Cod javascript pentru calcularea primelor 5000 de numere prime:

```
class AcceptFilter {
   accept(n) {
      return true
   }
}

class DivisibleByFilter {
   constructor(number, next) {
      this.number = number;
      this.next = next;
   }

   accept(n) {
      var filter = this;
      while (filter != null) {
         if (n % filter.number === 0) {
            return false;
         }
         filter = filter.next;
      }
      return true;
   }
}
```

```
class Primes {
   constructor() {
      this.number = 2;
      this.filter = new AcceptFilter();
   }

   next() {
      while (!this.filter.accept(this.number)) {
         this.number++;
      }
      this.filter = new DivisibleByFilter(this.number, this.filter);
      return this.number;
   }
}

var primes = new Primes();
var primesArray = [];
for (let i = 0; i < 5000; i++) {
   primesArray.push(primes.next());
}
console.log(`Computed ${primesArray.length} prime numbers. ` +
`The last 5 are ${primesArray.slice(-5)}.`);
```

Pentru a executa codul de mai sus, deschideți un terminal în directorul cu fișierul javascript și executați comanda de mai jos, modificând calea până la graalvm corespunzător:

```
~/graalvm-ce-java11-22.0.0.2/bin/js primes.js
```

În funcție de parametrii pe care îi adăugați la comanda de mai sus, veți vedea detalii cu privire la:

- utilizarea CPU-ului, defalcată pe funcții (--cpusampler)

```
-> ~/graalvm-ce-java11-22.0.0.2/bin/js primes.js --cpusampler
Computed 5000 prime numbers. The last 5 are 48563,48571,48589,48593,48611.
-------------------------------------------------------------------------------------------
Sampling Histogram. Recorded 203 samples with period 10ms. Missed 0 samples.
  Self Time: Time spent on the top of the stack.
  Total Time: Time spent somewhere on the stack.
-------------------------------------------------------------------------------------------
Thread[main,5,main]
 Name       ||             Total Time    ||              Self Time    || Location
-------------------------------------------------------------------------------------------
 accept     ||             2000ms  98.5% ||             2000ms  98.5% || primes.js~13-22:191-419
 next       ||             2030ms 100.0% ||               30ms   1.5% || primes.js~31-37:537-737
 :program   ||             2030ms 100.0% ||                0ms   0.0% || primes.js~1-46:0-970
-------------------------------------------------------------------------------------------
```

- utilizarea CPU-ului cu filtru pe denumirile funcțiilor (--cpusampler --cpusampler.FilterRootName=*accept). În acest caz, toate funcțiile care se termină cu „accept”.

```
->~/graalvm-ce-java11-22.0.0.2/bin/js primes.js --cpusampler --cpusampler.FilterRootName=*accept
Computed 5000 prime numbers. The last 5 are 48563,48571,48589,48593,48611.
-------------------------------------------------------------------------------------------
Sampling Histogram. Recorded 197 samples with period 10ms. Missed 0 samples.
  Self Time: Time spent on the top of the stack.
  Total Time: Time spent somewhere on the stack.
-------------------------------------------------------------------------------------------
Thread[main,5,main]
 Name       ||             Total Time    ||              Self Time    || Location
-------------------------------------------------------------------------------------------
 accept     ||             1970ms 100.0% ||             1970ms 100.0% || primes.js~13-22:191-419
-------------------------------------------------------------------------------------------
```

- numărul de executări ale unor instrucțiuni dintr-o funcție (--cputracer --cputracer.TraceStatements --cputracer.FilterRootName=*accept)

```
src : bash — Konsole
->~/graalvm-ce-java11-22.0.0.2/bin/js primes.js --cputracer --cputracer.TraceStatements --cputracer.FilterRootName=*accept
Computed 5000 prime numbers. The last 5 are 48563,48571,48589,48593,48611.
-----------------------------------------------------------------------------------------
Tracing Histogram. Counted a total of 468336895 element executions.
  Total Count: Number of times the element was executed and percentage of total executions.
  Interpreted Count: Number of times the element was interpreted and percentage of total executions of this element.
  Compiled Count: Number of times the compiled element was executed and percentage of total executions of this element.
-----------------------------------------------------------------------------------------
 Name     |          Total Count |    Interpreted Count |       Compiled Count | Location
-----------------------------------------------------------------------------------------
 accept   |     234117338  50.0% |         69512   0.0% |     234047826 100.0% | primes.js~15:245-258
 accept   |     117053670  25.0% |         34640   0.0% |     117019030 100.0% | primes.js~16-18:275-348
 accept   |     117005061  25.0% |         33995   0.0% |     116971066 100.0% | primes.js~19:362-381
 accept   |         53608   0.0% |           761   1.4% |         52847  98.6% | primes.js~14:211-227
 accept   |         53608   0.0% |           761   1.4% |         52847  98.6% | primes.js~13-22:191-419
 accept   |         48609   0.0% |           645   1.3% |         47964  98.7% | primes.js~17:322-334
 accept   |          4999   0.0% |           116   2.3% |          4883  97.7% | primes.js~21:402-413
 accept   |             1   0.0% |             1 100.0% |             0   0.0% | primes.js~3:45-55
 accept   |             1   0.0% |             1 100.0% |             0   0.0% | primes.js~2-4:25-61
-----------------------------------------------------------------------------------------
```

- alocările de memorie (--experimental-options --memtracer)

```
src : bash — Konsole
->~/graalvm-ce-java11-22.0.0.2/bin/js primes.js --experimental-options --memtracer
Computed 5000 prime numbers. The last 5 are 48563,48571,48589,48593,48611.
-------------------------------------------------------
 Location Histogram with Allocation Counts. Recorded a total of 5007 allocations.
   Total Count: Number of allocations during the execution of this element.
   Self Count: Number of allocations in this element alone (excluding sub calls).
-------------------------------------------------------
 Name     |      Self Count |     Total Count | Location
-------------------------------------------------------
 next     |     5000  99.9% |     5000  99.9% | primes.js~31-37:537-737
 :program |        6   0.1% |     5007 100.0% | primes.js~1-46:0-970
 Primes   |        1   0.0% |        1   0.0% | primes.js~25-38:424-739
-------------------------------------------------------
```

De asemenea, GraalVM oferă posibilitatea de depanare prin ataşarea unui debugger precum Chrome Developer Tools.

Pentru mai multe detalii, vezi:

- https://www.graalvm.org/tools/profiling/
- https://www.graalvm.org/tools/chrome-debugger/
- https://www.graalvm.org/tools/visualvm/

Aplicaţii şi teme

Aplicaţii de laborator:

1. Schimbaţi algoritmul de calcul al sumei de control, scris în **Python**, folosind o formulă polinomială de cel mult rang 5.
2. Pentru calculul sumei de control, să se elimine primul şi ultimul caracter folosind funcţii substring.
3. Să se implementeze o aplicaţie Polyglot într-un limbaj la alegere (suportat de GraalVM), care să conţină trei funcţii:
   - o funcţie **python** care să genereze aleatoriu o listă de 20 de numere întregi;
   - o funcţie **javascript** care va afişa lista de numere întregi;
   - o funcţie **R** care să sorteze lista respectivă, să elimine primele şi ultimele 20% dintre elemente şi să calculeze media aritmetică.

   Rezultatul final va fi afişat la consolă. Se poate pleca de la scheletul aplicaţiei exemplu.

Teme pe acasă:

1. Modificaţi aplicaţia de laborator (după ce s-au implementat cerinţele acestuia) astfel încât să afişeze toate cuvintele cu aceeaşi sumă de control.
2. Să se implementeze o regresie liniară utilizând limbajul **R** pe parte statistică, cu posibilitatea introducerii setului de date din python/java/orice alt limbaj cu care sunteţi familiari (şi este suportat de GraalVM). Rezultatul plotării (imaginea) va fi salvat pe disk şi va fi deschis apelând o funcţie sistem. Întrucât

GraalVM nu oferă încă suport pentru majoritatea bibliotecilor grafice, se va folosi biblioteca ***lattice*** pentru plotare. Setul de date de intrare va fi introdus într-un fișier numit *dataset.txt*, fiind citit de aplicația master și trimis ca parametru funcției de regresie liniară. De asemenea, se vor citi de la tastatură numele fișierului de output (numele imaginii), calea unde se dorește să fie salvat și culorile de plotare.

3. Să se implementeze o aplicație Polyglot într-un limbaj la alegere (suportat de GraalVM) care să citească prin intermediul unei funcții **python** două numere naturale de la tastatură: numărul de aruncări ale unei monede și un alt număr x, unde $1 \leq x \leq$ numărul de aruncări. Aplicația va utiliza capabilitățile statistice ale limbajului **R** pentru a calcula distribuția binomială corespunzătoare aruncării unei monede și va afișa probabilitatea de a obține de cel mult x ori pajură din numărul total de aruncări.